

SQL Advanced Joins & Subqueries

USING Clause

The USING clause is a shorthand for JOIN ... ON when both tables have a column with the same name.

Syntax

Using ON

```
SELECT
    e.name,
    d.department_name
FROM Employee e
JOIN Department d
ON e.department_id = d.department_id;
```

Using USING

```
SELECT
    e.name,
    d.department_name
FROM Employee e
JOIN Department d
USING (department_id);
```

Both produce the same result.

Difference Between USING and ON

ON

```
SELECT *
FROM A
JOIN B
ON A.id = B.id;
```

Result

id	...	id	...
----	-----	----	-----

Both columns remain.

USING

```
SELECT *  
FROM A  
JOIN B  
USING(id);
```

Result

id	...
----	-----

Only one copy of the common column appears.

Multiple Columns

```
SELECT *  
FROM Sales s  
JOIN Prices p  
USING(product_id, year);
```

Equivalent to

```
ON s.product_id = p.product_id  
AND s.year = p.year
```

Limitations

✗ Column names must be identical.

Wrong: `student_id` vs `id` - Cannot use USING.

Must write:

```
ON s.student_id = e.id
```

Joins with Aggregation

One of the most important SQL interview topics.

Example 1 — Orders per Customer

Tables: Customers ↓ Orders

Query:

```
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS total_orders
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.name;
```

Example 2 — Revenue per Category

```
SELECT
    c.category_name,
    SUM(oi.quantity * oi.price) AS revenue
FROM Categories c
JOIN Products p
ON c.category_id = p.category_id
JOIN OrderItems oi
ON p.product_id = oi.product_id
GROUP BY c.category_name;
```

```
SELECT
    c.course_name,
    COUNT(e.student_id) AS total_students
FROM Courses c
LEFT JOIN Enrollment e
ON c.course_id = e.course_id
GROUP BY c.course_name;
```

List the Products Ordered in a Period

```
SELECT
    p.product_name,
    SUM(o.unit) AS unit
FROM Products p
JOIN Orders o
ON p.product_id = o.product_id
WHERE o.order_date BETWEEN '2020-02-01' AND '2020-02-29'
GROUP BY p.product_name
HAVING SUM(o.unit) >= 100;
```

Joins with Subqueries

Instead of joining directly to a table, you join to the result of another query.

Derived Table

Example

```
SELECT *
FROM
(
    SELECT
        customer_id,
        COUNT(*) AS total_orders
    FROM Orders
    GROUP BY customer_id
) t;
```

The subquery behaves like a temporary table.

Example — Highest Salary Per Department

```
SELECT
    e.name,
    e.department,
    e.salary
FROM Employee e
JOIN
(
    SELECT
        department,
        MAX(salary) AS max_salary
    FROM Employee
    GROUP BY department
) t
ON e.department = t.department
AND e.salary = t.max_salary;
```